

Chapter 60. Designing Whole Systems

Draw the whole picture, turn adjectives into numbers, and write decisions down before the meeting is forgotten.

Approved chapter v1

Contents

Chapter 60. Designing Whole Systems

Why this matters

In plain English

60.1 What "architecture" means, practically

60.2 Reading and drawing C4 diagrams

60.3 Non-functional requirements: numbers, not adjectives

60.4 Trade-offs and the architecture decision record

60.5 The worked example: designing the Riverstone Analytics & AI Platform

60.6 Evolving a design under real constraints

Common mistakes and how to spot them

In the real world: the diagram that almost caused a rewrite

Tools

The project: design document for Riverstone's platform

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 60. Designing Whole Systems

Part VII — Architecture, Governance & Leadership

Chapter at a glance

You will learn to: explain what "architecture" means once a decision affects more than one team's tools · draw and read a C4 diagram at the zoom level a conversation actually needs · turn a vague wish ("make it fast", "make it secure") into a number someone can test against · write an architecture decision record (ADR) that a stranger could read in two years and understand why · design a whole system by composing the pieces earlier parts of this book built separately · evolve a design under real constraints — budget, headcount, a legacy system nobody is allowed to touch · recognize the failure patterns of over-engineering, resume-driven design, and the big-bang rewrite.

Before you start: this chapter assumes you've read enough of the book to recognize the pieces it assembles: the warehouse and orchestration from Part V, the AI systems from Part VI, and the dashboards and automations from Part II. You don't need to remember every detail — each piece is reintroduced in one line before it's used.

Time needed: 12–15 hours, spread over two weeks.

Tools: nothing new to install. Diagrams in this chapter are drawn with a free tool (Mermaid, diagrams.net, or even a whiteboard); the companion files show the same diagrams as plain text so you can redraw them yourself.

Practice data: `companion/ch60/` : the full worked design document for a Riverstone Analytics & AI Platform, five architecture decision records, and a non-functional-requirements worksheet — all built from the real systems described in Parts II, V, and VI of this book.

Why this matters

Every chapter before this one taught you to build one good thing: a clean pipeline, a trained model, a dashboard that doesn't lie. Riverstone, over the course of this book, now has all of them — a warehouse, an orchestrator, a defect-detection model, a document assistant, an email-reading pipeline that writes into the ERP, a daily flash report. Each one, on its own, works.

Put them in the same company at the same time and a new kind of problem appears, one that no single chapter could have taught, because it isn't about any one piece. It's about the *seams*: what happens when the orchestrator's nightly run and the PO-intake pipeline's hourly run both want to write to the same order table at 6:03 a.m.? Who gets paged when the defect model's serving endpoint goes down — the person who built the model, or the person who owns the pipeline that feeds it? If the CRM sync and the support assistant both need the customer's current address, which one is allowed to be wrong for five minutes?

Architecture is the discipline of answering those questions on purpose, before they happen, instead of by accident, at 2 a.m., in production. It's not a fancier word for "system design" reserved for people with the word "Architect" in their title — it's what any senior analyst or engineer starts doing the moment their decisions stop affecting only their own script and start affecting everyone else's.

This chapter teaches the vocabulary and the habits: how to draw a system so someone else can understand it, how to turn "make it good" into something you can actually check, how to write down a decision so it doesn't get re-litigated every six months, and how to look at eight separately-built Riverstone systems and describe them as one platform.

In plain English

Think about the difference between building one excellent room in a house and being the person who decides where the plumbing goes for the whole house.

A brilliant kitchen, on its own, doesn't need an architect. But the moment there's a kitchen, two bathrooms, and a laundry room, someone has to decide: which rooms share a wall so the pipes can be shared too? What happens if the water pressure drops when three taps run at once? If the kitchen is remodeled next year, which walls can move without the upstairs bathroom falling through the floor?

Nobody asks these questions while admiring one perfect room. They come from stepping back and looking at the whole house — and from having been the person who, at 2 a.m., got the call that the upstairs bathroom is now the downstairs kitchen's ceiling.

That step-back view — the whole house, not one room — is what this chapter, and this Part of the book, is about. The individual rooms (the pipelines, the models, the dashboards) were Parts II through VI. Part VII is the plumbing diagram, the load-bearing walls, and the decision about which walls can move.

60.1 What "architecture" means, practically

Architecture is any decision that's expensive to change later and affects more than one part of the system. That's a working definition, not a formal one, and it's useful because it tells you exactly when you're doing architecture and when you're not.

Choosing a variable name is not architecture — it costs nothing to change and affects one function. Choosing Dagster as Riverstone's orchestrator (Chapter 46) *was* architecture — every pipeline, every team, and every future hire now has to know Dagster, and un-choosing it means rewriting all of them. Choosing to store the sensor archive as Delta Lake instead of plain Parquet files (Chapter 49) was architecture for the same reason: it's a decision that would be genuinely painful to reverse once fifty pipelines depend on it.

Three properties usually mark a decision as architectural:

1. **It's hard to reverse.** Not impossible — nothing is impossible — but expensive enough that "we'll just change it later" is not a credible plan.
2. **It constrains what comes after it.** Once the warehouse is Postgres, every later choice (which BI tool, which orchestrator, which language for pipelines) has to work with Postgres.
3. **Getting it wrong is expensive in a way that shows up late.** A bad architecture decision often looks fine for months, then fails exactly when the system is under the most load or the most scrutiny — the pattern behind nearly every "war story" in Chapters 45 through 58.

What an architect actually does, day to day, is less glamorous than the title suggests and more useful than most people expect:

- Asks "what happens when this fails?" before it's built, not after.
- Draws the system so the five people who each understand one piece of it can see how their pieces fit.
- Says no to a request that would work today and become unmaintainable in a year — and can explain why, in a way the requester accepts.
- Writes decisions down, so the next person doesn't have to reverse-engineer *why* from the code.

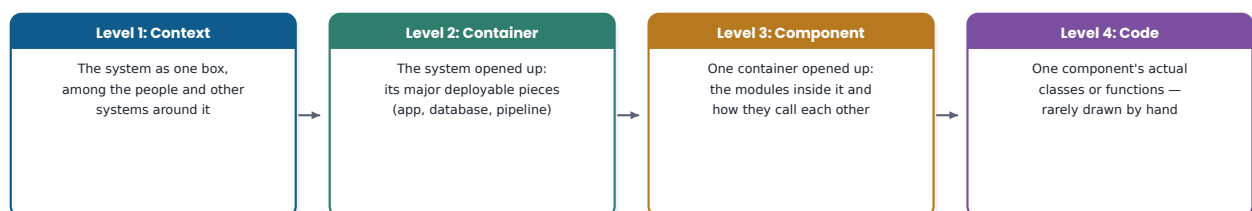
None of that requires a special title. Every analyst who's chosen a database schema, every engineer who's decided where a check belongs, has already been doing architecture in miniature. This Part scales the same judgment up to the level of an entire company's systems.

60.2 Reading and drawing C4 diagrams

The most common failure in system communication isn't a bad diagram — it's a diagram at the wrong **zoom level** for the conversation. Someone asks "how does data get from the plant to the dashboard?" and receives a diagram with forty labeled function calls, or asks "what does this one service do internally?" and receives a single box labeled "Backend."

The **C4 model** (Context, Containers, Components, Code), from software architect Simon Brown, fixes this by naming four fixed zoom levels and insisting you draw one at a time.

The C4 model: the same system, four levels of zoom

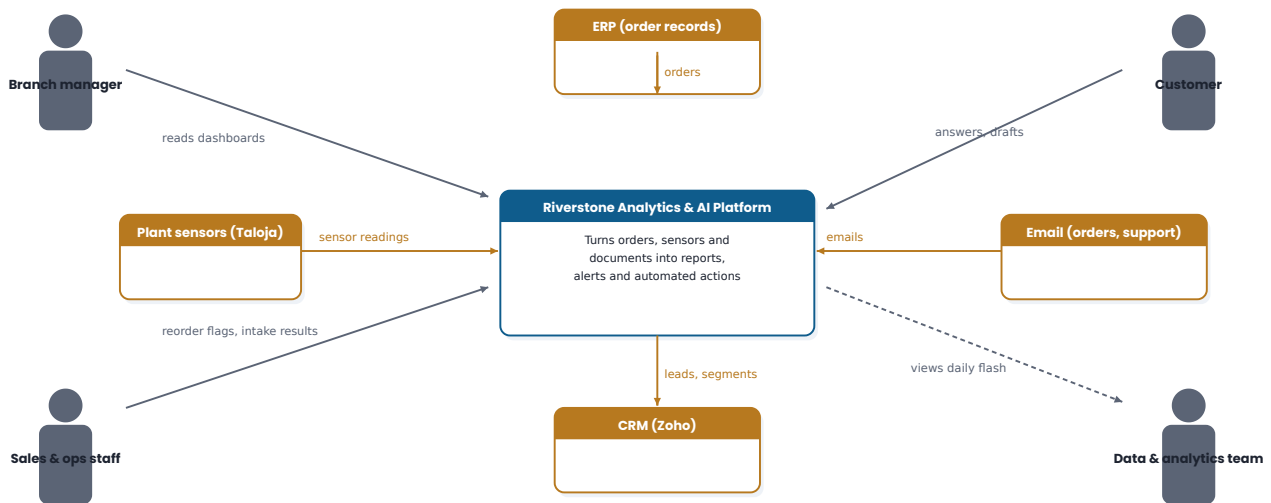


Zoom in one level at a time. A context diagram with component-level detail crammed in helps nobody; a component diagram of the whole company is unreadable. Pick the zoom level the conversation needs, and stop there.

Figure 60.1 — Zoom in one level at a time. Mixing levels — a context diagram with implementation detail crammed in — helps nobody.

Level 1: Context. The system is one box. Everything else is either a person (a user) or another system it talks to. This is the diagram you show someone who needs to know *what the system is for*, not how it works.

Context diagram: the Riverstone Analytics & AI Platform, level 1



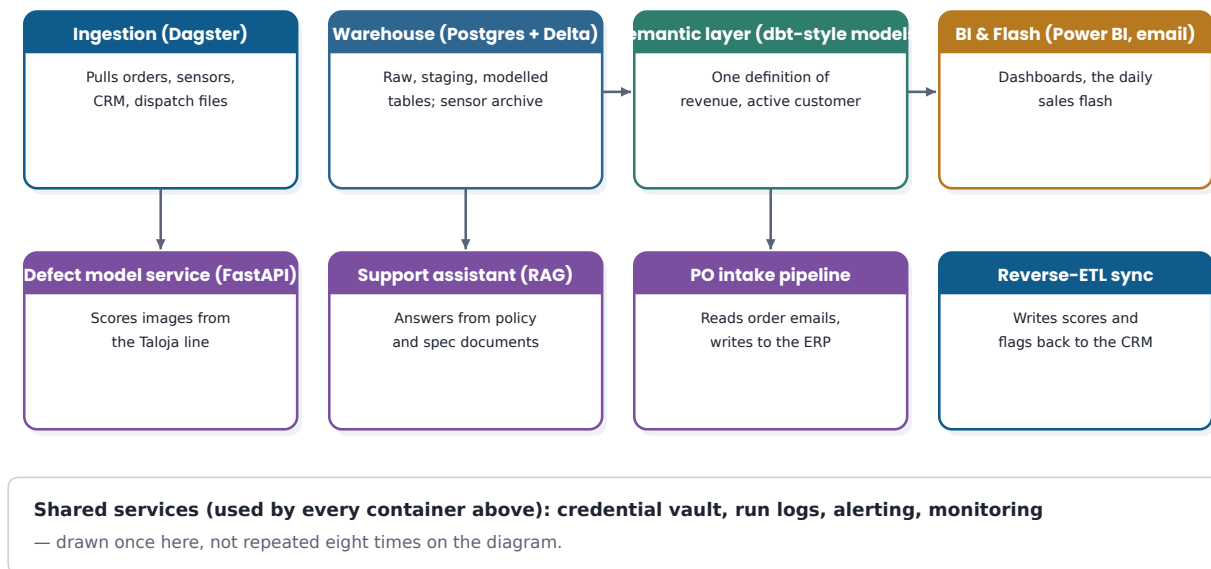
One box in the middle; every arrow crosses its boundary. At this zoom level, nobody asks which orchestrator runs inside — that's level 2.

Figure 60.2 — A context diagram for the platform this chapter designs. One box, and every arrow crosses its boundary. Nobody here asks which orchestrator runs inside — that question belongs one level down.

At this level, a new employee, a manager, or an auditor can understand the whole picture in thirty seconds: the platform reads orders from the ERP and Zoho, reads sensor data from the plant, reads and answers emails, and is read by dashboards, alerts, and the sales team's own tools. Nothing about *how* appears yet, and that's the point.

Level 2: Container. Open the one box from level 1 and show its major deployable pieces — the things you could run, scale, or replace independently. "Container" here doesn't mean Docker specifically (though a container in this sense is often deployed as one); it means a separately runnable unit.

Container diagram: opening the platform box, level 2



Each box is something you could deploy and scale on its own. This is the level where you decide what talks to what, and how.

Figure 60.3 – Opening the context box. This is the level where "what talks to what, and how" gets decided – and where most of this book's later chapters live.

Notice what the container diagram makes visible that the context diagram hid: there are **four AI-adjacent containers** (the defect model, the support assistant, the PO-intake pipeline, and the reverse-ETL sync) sitting alongside the four data-platform containers from Part V. A reader who only saw the context diagram would have no idea the system had grown this complex inside; a reader who only saw a component-level diagram of one container would have no idea it was one piece of eight.

Level 3: Component. Open one container and show the modules inside it and how they call each other – for example, the PO-intake pipeline's read/extract/parse/validate/decide/write/report stages from Chapter 58. You draw this level for the team that owns that one container, not for the whole company.

Level 4: Code. The actual classes or functions. The C4 method's own advice, and this book's: **don't draw this by hand**. Generate it from the code if a tool can, or skip it – a UML class diagram maintained manually goes stale within a month and actively misleads after that.

The discipline that matters more than the notation: before drawing anything, ask *who is this for, and what do they need to decide?* A board member needs level 1. A new hire on the data platform team needs level 2, and level 3 for whichever container they'll work on first. Nobody, ever, needs level 4 drawn by hand.

60.3 Non-functional requirements: numbers, not adjectives

A **functional requirement** says what the system does: "send the daily flash email," "score an image for defects." A **non-functional requirement (NFR)** says how well it has to do it: how fast, how available, how secure, how much it may cost. Most systems fail not because the functional requirements were

wrong, but because the non-functional ones were never written down, so nobody could tell whether the system met them.

The problem is that non-functional requirements almost always arrive as adjectives — "fast," "reliable," "secure," "scalable" — and adjectives can't be tested. **The job of this section is turning each adjective into a number.**

Turning a vague requirement into a number

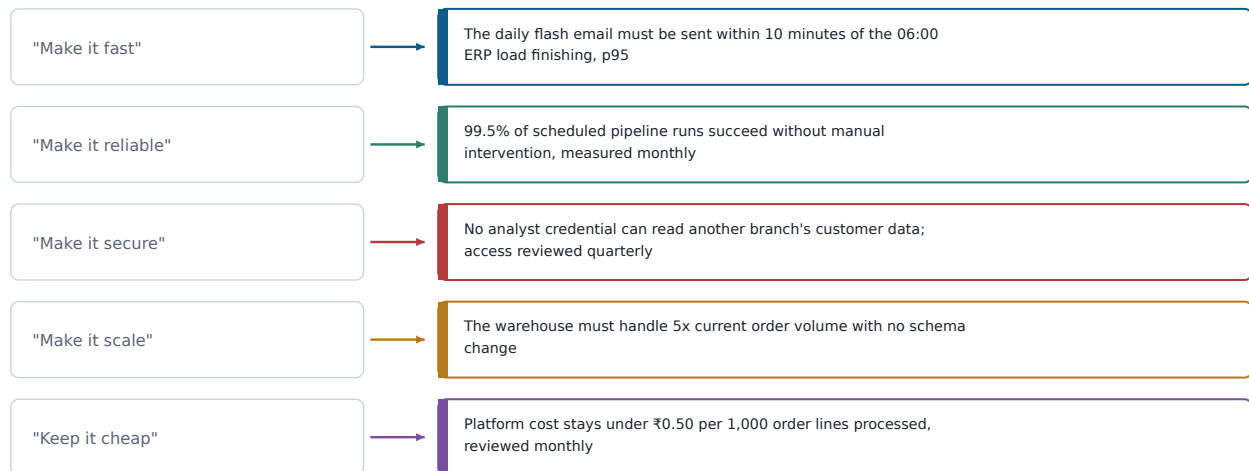


Figure 60.4 — Every one of these vague requirements was actually said, in some form, about a real Riverstone system in Parts II, V, or VI. The right-hand column is what an architect writes down instead.

A template for turning any adjective into a requirement:

Adjective	Ask	Turns into
Fast	How fast, measured how, for what share of requests?	"p95 latency under 200ms"
Reliable	What fraction of the time, measured over what window?	"99.5% of scheduled runs succeed, measured monthly"
Secure	Secure against what, specifically?	"No credential can read data outside its own branch"
Scalable	To what future volume, by when?	"Handles 5x current order volume with no schema change"
Cheap	Compared to what, and is this a ceiling or a target?	"Under ₹0.50 per 1,000 order lines processed"

Two categories are worth calling out because they're the ones systems most often get wrong silently:

- **Availability** is usually expressed in "nines" — 99% availability is about 3.65 days of downtime a year; 99.9% is about 8.75 hours; 99.99% is about 53 minutes. Each additional nine costs disproportionately more to achieve, which is exactly why an architect's job includes asking *does this system actually need 99.99%*, not assuming more is always better. Riverstone's daily flash email genuinely doesn't need four nines; a plant safety alert genuinely might.
- **Consistency and freshness** — "how stale can this number be before it's wrong" — is a non-functional requirement people forget to state and then argue about after the fact. Chapter 47's

freshness checks exist precisely because "the data should be current" was never turned into "orders are no more than 2 hours stale, checked hourly."

Watch out: an NFR with no owner and no test is a wish, not a requirement. Writing "99.5% uptime" in a document and never measuring it against reality is worse than not writing it at all, because it creates false confidence. Every NFR in this chapter's worked example (section 60.5) has a stated measurement method.

60.4 Trade-offs and the architecture decision record

Almost no architectural choice is strictly better than its alternative — if it were, there'd be nothing to decide. Every real choice **trades one good property for another**: Delta Lake trades operational simplicity (one more tool to run) for reliability (atomic writes, time travel) that plain Parquet files don't have. Dagster trades a learning curve for testability and observability that a folder of cron jobs doesn't have. There is no version of this book, or this platform, where every choice is free.

An architecture decision record (ADR) is a short, permanent note that captures one such choice: what was decided, why, what else was considered, and what the consequences are. Its value isn't in the decision itself — it's in **making the reasoning outlive the meeting where it was made**.

An architecture decision record, filled in

ADR-014	Table format for the sensor archive
Status	Accepted, 18 Sep 2026
Context	The Taloja sensor archive is 19.8M+ rows and growing daily. A six-week correction to machine M-07's readings failed mid-run under plain Parquet files, leaving the archive in a partly-corrected, unrecoverable state for two days.
Decision	Store the sensor archive as a Delta Lake table (delta-rs), not plain Parquet files.
Alternatives considered	(1) Plain Parquet with manual versioning by folder name — rejected: exactly what failed. (2) Apache Iceberg — comparable guarantees, rejected only because the team has no existing Java/Spark operational experience. (3) Keep it in Postgres — rejected: cost and query performance at this row count.
Consequences	Atomic overwrites and time travel (Ch49); one more format for the team to operate; the `deltalake` Python package must be pinned and tested on version upgrades.
Owner	Data platform team. Revisit if Iceberg adoption becomes a company standard.

Figure 60.5 — One real decision from Part V, written up properly. Two years from now, nobody has to guess why the sensor archive uses a format the rest of the warehouse doesn't.

The six parts of an ADR:

1. **Title and number** (ADR-014), so it can be referenced from code, tickets, or other ADRs.
2. **Status** (proposed, accepted, superseded) and the date.
3. **Context** — the situation that made a decision necessary. Chapter 49's context wasn't hypothetical: a real correction run failed mid-way under plain files and cost two days of recovery.
4. **Decision** — one sentence, stated plainly.
5. **Alternatives considered**, each with why it was rejected. This is the part people skip, and it's the part that prevents the same debate from happening again in eight months when someone new

joins and asks "why don't we just use Iceberg?"

6. **Consequences** — including the costs of the decision, not only its benefits. Every decision has a cost; naming it is what makes the ADR honest rather than promotional.

When to write one: any time a decision meets section 60.1's test — hard to reverse, affects more than one part of the system, or expensive to get wrong. Not every choice needs one; a folder of a hundred ADRs for trivial decisions is as useless as none at all. Riverstone's data platform team, by the time Chapter 60's design document was written, had accumulated fourteen — one for the orchestrator, one for the table format, one for the cloud region, and so on, each traceable to a specific chapter's story.

60.5 The worked example: designing the Riverstone Analytics & AI Platform

Everything so far has been vocabulary. This section uses it on a real design problem: by early 2026, Riverstone has built a warehouse and orchestrator (Part V), a defect-detection model, a document assistant, and an email-reading order pipeline (Part VI), and a set of dashboards and automations (Part II) — separately, by different people, at different times, for different immediate needs. Nobody has ever drawn what all of it looks like *together*, and two incidents in one month (a pipeline collision at 6 a.m., and a support-assistant answer that quoted a superseded policy the RAG system happened to rank first) made clear that "together" is now a real system, whether anyone designed it as one or not.

The design document below is what an architect produces in that situation — not to invent new systems, but to describe, constrain, and connect the ones that already exist. The full version, with every section expanded, is `companion/ch60/design-document.md` ; what follows is the document's spine.

Purpose and scope

One paragraph, always first, always readable by someone with no context. "This document describes the Riverstone Analytics & AI Platform: the combined set of data pipelines, models, and delivery systems that turn order, sensor, and document data into reports, alerts, and automated actions across sales, operations, and support. It does not cover the ERP or CRM themselves, which remain systems of record owned by their respective vendors and teams."

Requirements

Functional, in one line each: ingest orders, sensor readings, and CRM data reliably; detect line-level product defects from images; answer support and policy questions from current documents only; read purchase-order emails and write validated orders to the ERP; deliver a daily sales summary; keep the CRM's lead scores current.

Non-functional, using section 60.3's method:

Requirement	Precise form	Measured how
Freshness	Orders no more than 2 hours stale during business hours	Automated check, hourly (Ch 47)

Requirement	Precise form	Measured how
Pipeline reliability	99.5% of scheduled runs succeed without manual intervention	Monthly, from Dagster run history (Ch 46)
Flash delivery	Sent within 10 minutes of the 06:00 ERP load finishing, p95	Automated timestamp comparison (Ch 20, Ch 46)
Defect model latency	p95 under 50ms per image, at line speed	Load test before each deployment (Ch 56)
Assistant grounding	Refuses rather than guesses when no supporting document scores above the floor	Golden-set evaluation, every deploy (Ch 55, Ch 57)
PO-intake accuracy	No order auto-loads without either high extraction confidence or human confirmation	Per the Ch 58 finding: assisted, not straight-through
Cost ceiling	Platform cost under ₹0.50 per 1,000 order lines processed	Monthly cost review (Ch 65)
Access control	No credential can read another branch's customer data	Quarterly access review (Ch 64)

The architecture: context and containers

Figure 60.2 (context) and Figure 60.3 (container) are this section — a design document's diagrams are not illustrations of the text, they're load-bearing parts of it. The prose around them explains what isn't obvious from the boxes:

- **Ingestion, warehouse, and semantic layer are shared infrastructure** that every other container reads from or writes to. They are the one part of the platform that, if it goes down, everything else eventually stops being trustworthy.
- **The four AI-adjacent containers are independent of each other.** The defect model doesn't know the support assistant exists, and neither depends on the PO-intake pipeline. This is deliberate: a failure in one shouldn't cascade into the others, and each was built and can be redeployed on its own schedule.
- **Two containers write to systems outside the platform's control:** the PO-intake pipeline writes orders to the ERP, and the reverse-ETL sync writes scores to the CRM. Both are held to a stricter standard than read-only containers — idempotency, a named system of record per field, and reconciliation (Chapter 51's discipline) — because a wrong write has consequences a wrong read doesn't.

Key decisions (the ADR index)

A real design document doesn't re-argue each decision inline; it links to the ADR and states the conclusion. The platform's index, abbreviated:

ADR	Decision	Chapter
ADR-002	Orchestrator: Dagster	46
ADR-007	Cloud region: AWS ap-south-1 (Mumbai)	52
ADR-014	Sensor archive table format: Delta Lake	49

ADR	Decision	Chapter
ADR-018	Quality tooling: SQL tests in CI, not a dedicated platform (yet)	47
ADR-021	PO-intake mode: assisted, not straight-through, given a 19% silent-error rate at full automation	58
ADR-023	Support assistant: absolute confidence floor for refusal, not a normalized one, after a scoring bug ranked a superseded policy first	55
ADR-026	CRM writes: PATCH only, one system of record per field, after a prior sync's PUT erased a negotiated discount	51

Reading down this table is, itself, a compressed history of Parts V and VI: almost every decision traces to an incident, a measurement, or a finding that a simpler approach didn't hold up. **That's what a healthy ADR index looks like** — not a list of things that sounded good, but a list of things that were tested against reality and survived.

Data flow: one request, traced end to end

Design documents earn their keep when they can answer "what actually happens when—?" Trace one: *a customer emails a purchase order*.

1. The PO-intake pipeline (a Dagster asset) picks up the email within its polling interval.
2. Extraction runs; if confidence is high and validation passes, the order is queued for one-click confirmation (Chapter 58's assisted mode); otherwise it's routed to the exception queue.
3. On confirmation, the order is written to the ERP with an idempotency key derived from the source email, so a re-run or a retry can never create a duplicate (Chapter 45's and Chapter 51's pattern, applied here).
4. The next scheduled ingestion run picks up the new order from the ERP into the warehouse.
5. The semantic layer's models recompute affected aggregates.
6. The daily flash, dashboards, and the reverse-ETL sync to the CRM all read from the same, now-updated, semantic layer — so the sales rep's CRM record and the branch manager's dashboard agree, because they share one upstream source rather than being computed twice.

Nowhere in that chain does one container reach into another's internals. Every arrow in the container diagram is a real, traceable step in this walkthrough — which is exactly the test a container diagram should pass: **if you can't trace a real request across it, the diagram is decoration, not documentation.**

Risks and open questions

A design document that only lists what's decided is marketing. It should also say what isn't:

- **The PO-intake exception queue has no defined maximum wait time.** If email volume triples during a festive season, does the human-confirmation step become the bottleneck? Not yet load-tested.
- **The defect model and the support assistant share no infrastructure today,** which is good for isolation and bad for cost — two separate serving setups for two low-traffic models. Worth

revisiting once a third model exists.

- **Chapter 64 (this Part) has not yet defined the platform's access-control model**, so the "no credential can read another branch's data" requirement above is aspirational until that chapter's work lands.

60.6 Evolving a design under real constraints

The design in section 60.5 describes what Riverstone's platform *should* look like. Getting there from eight separately-built systems, on a real budget, with a small team, is a different problem — and it's the one architects spend most of their time on, because greenfield designs are rare and everything else is evolution.

Three constraints shape almost every real redesign:

- **Budget.** The "right" answer (a dedicated feature store, a managed observability platform, a second cloud region for redundancy) often costs more than the problem it solves is worth. Part of the job is sizing the fix to the pain, not to the textbook.
- **Headcount.** A four-person data team cannot operate the same number of moving parts as a forty-person one, no matter how good the architecture diagram looks. Fewer, more standardized containers usually beat more, more specialized ones, for a small team.
- **Legacy systems nobody can touch.** The ERP is a vendor system; Riverstone can integrate with it but not redesign it. Good architecture works *around* that constraint openly (as the PO-intake pipeline does, treating the ERP as a system of record it writes to carefully) rather than pretending the constraint doesn't exist.

A pattern for evolving safely, used throughout Parts V and VI without always naming it:

1. **Run the old and new side by side** before switching (Chapter 56's shadow-mode comparison of two model versions is this pattern applied to a model; the same idea applies to a new pipeline or a new table format).
2. **Migrate the lowest-risk piece first**, to learn the real cost of the change before betting something important on it.
3. **Keep a rollback path** until the new piece has run through at least one full cycle of whatever could go wrong (a month-end close, a festive season, a provider outage).
4. **Write the ADR before, not after** — deciding under pressure, mid-migration, is how the alternatives-considered section gets forgotten.

Resisting the urge to redesign everything at once is itself a skill. The temptation, once you can see the whole system clearly for the first time, is to fix all of it — new orchestrator, new warehouse, new everything, all at once, on the strength of one good diagram. This almost never survives contact with a real company's calendar, and it's the subject of this chapter's mistakes table below.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Drawing the wrong zoom level	A board member glazes over at a component diagram; an engineer can't build from a context diagram	Ask who it's for and what they need to decide, before drawing anything
Treating every decision as architectural	A hundred ADRs, most for trivial choices, that nobody reads	Reserve ADRs for decisions that are hard to reverse or cross team boundaries
Non-functional requirements left as adjectives	"Make it fast" with no way to know if it succeeded	Turn every adjective into a number and a measurement method
An NFR with no owner	A requirement nobody checks, quietly false for months	Name who measures it and how often
Skipping "alternatives considered" in an ADR	The same debate happens again in eight months	Always record what was rejected, and why
A container diagram nobody can trace a real request through	The diagram is decorative, not documentation	Walk one real request across it before calling it done
Over-engineering for a scale that doesn't exist yet	Months spent on redundancy a four-person team can't operate	Size the design to the current constraint, with a documented path to grow
Resume-driven design	A new tool adopted because it's exciting, not because the old one failed	Require a real, named problem the new tool solves that the old one doesn't
The big-bang rewrite	Months of work, nothing shippable, and the old system rots waiting	Migrate the lowest-risk piece first; run old and new side by side
Ignoring legacy constraints	A design that assumes the ERP can be redesigned, when it can't	Design around real constraints, stated openly, not around a wish list
No rollback path during migration	A change that can't be undone if it goes wrong mid-flight	Keep the old path live until the new one has survived a full cycle
Deciding under pressure, mid-incident	The ADR gets written after the fact, missing the real alternatives	Write the ADR before the change, even a short one, not after

In the real world: the diagram that almost caused a rewrite

By January 2026, Meera Iyer had spent four years building pieces of what was now, without anyone having planned it, a genuine platform: the warehouse and Dagster pipelines from her data engineering work, the defect model and support assistant from the AI projects, the PO-intake pipeline that had quietly become the busiest thing she'd ever built. Anita Rao, watching the company's data work outgrow "the analytics team's scripts," asked Meera to take on a new title — Head of Data Platform — and to figure out what came next.

Meera's first act in the new role was the one this chapter recommends: she drew the container diagram. Not because a diagram would fix anything by itself, but because nobody — including her — had ever seen the whole thing laid out at once.

The diagram was, by her own later admission, "genuinely alarming." Eight containers, built at eight different times, by processes that had made sense individually and looked chaotic together. Two of them wrote to the ERP through different paths with different retry logic. The defect model and the support assistant each ran their own bespoke serving setup, duplicating infrastructure neither team had realized the other had built. The reverse-ETL sync and the CRM segment sync from Chapter 51 both touched customer records, with no agreed rule about which one won if they disagreed.

Her first instinct, looking at that diagram, was to propose exactly what section 60.6 warns against: a single unified platform, rebuilt properly from the ground up, over the next two quarters. She got as far as drafting the proposal before a conversation with Vikram Singh stopped her. His question was simple: *"What happens to the daily flash, the defect alerts, and the PO intake while you're rebuilding all of it?"*

She didn't have a good answer, because there wasn't one. A two-quarter rewrite meant two quarters of a frozen platform, or two parallel platforms to maintain, on a four-person team, while the business kept running on the messy-but-working version underneath.

What she did instead, over the following two months:

1. **She wrote ADR-026 first** — the CRM write-conflict rule (Chapter 51's `PATCH` -only, one-system-of-record-per-field decision) — because it was the one inconsistency actively causing wrong data, and it was fixable in a week without touching anything else.
2. **She consolidated the two model-serving setups** into one shared FastAPI service, migrating the lower-traffic one (the defect model) first, running it in shadow mode for two weeks before cutting over — exactly Chapter 56's pattern, applied to infrastructure rather than a model.
3. **She left the eight-container shape alone.** It wasn't elegant, but every container had a real owner and a real reason to exist, and nothing about the diagram's *shape* was actually causing the incidents. The incidents were caused by three specific, nameable gaps — the write conflict, the duplicated serving infrastructure, and an undocumented retry difference — not by the system's overall design.

The platform a year later looked almost the same as the "alarming" diagram from January, with three targeted fixes and fourteen ADRs explaining every non-obvious choice. Nobody remembers the rewrite proposal, because it never happened, and nothing was lost by not doing it.

What made the difference:

- **She drew the whole picture before deciding anything**, which turned a vague unease into three specific, fixable problems.
- **She asked what a full rewrite would cost while it was happening**, not only what it would achieve when finished.
- **She fixed what was actually broken**, not what merely looked untidy on a diagram — an untidy diagram of a working system is not the same problem as a broken one.

- **She wrote the decisions down**, so the next person to look at that container diagram would understand why it looks the way it does.
-

Tools

- **Diagramming:** Mermaid (text-based, renders in many tools including GitHub and this book's own published pages), diagrams.net (formerly draw.io, free and works offline), or a whiteboard photographed and cleaned up. No paid tool is required for anything in this chapter.
 - **ADR templates:** Michael Nygard's original ADR format (the one this chapter uses) is a short Markdown template; `adr-tools` is a free command-line helper for numbering and indexing them, if you want automation.
 - **Companion files (`companion/ch60/`):**
 - `design-document.md` : the full worked Riverstone Analytics & AI Platform design document, expanded from section 60.5.
 - `adr-template.md` : a blank, six-part ADR template.
 - `adr-examples/` : five filled-in ADRs from the platform's real index (orchestrator, cloud region, table format, PO-intake mode, CRM write rule).
 - `nfr-worksheet.md` : the adjective-to-number worksheet from section 60.3, blank and ready to fill in for your own system.
-

The project: design document for Riverstone's platform

Goal: produce a real design document — context diagram, container diagram, non-functional requirements, an ADR index, and a traced data flow — for a system you actually work with.

Option A: your own system. Pick something at your workplace that has grown past one person's or one team's full understanding — exactly the situation this chapter's worked example describes.

Option B: Riverstone. Extend section 60.5's document: draw the component-level diagram (level 3) for the PO-intake pipeline, using Chapter 58's read/extract/parse/validate/decide/write/report stages as your components.

Steps

1. **Write the purpose and scope paragraph first.** If you can't write it in three sentences a stranger would understand, the system isn't scoped clearly enough yet to design.
2. **List the functional requirements**, one line each.
3. **Turn at least five non-functional requirements from adjectives into numbers**, using section 60.3's table as a template, each with a stated measurement method and owner.
4. **Draw the context diagram** (level 1): one box, every person and system around it.
5. **Draw the container diagram** (level 2): the deployable pieces, and how they connect.

6. **Trace one real request end to end** across your container diagram, the way section 60.5 traced a purchase-order email. If you can't trace it, the diagram needs work.
7. **Write at least three ADRs** for real decisions already made (even informally) in this system, including the alternatives that were considered and rejected.
8. **List at least two open risks** honestly — something not yet solved, not yet load-tested, or not yet decided.

What good looks like: a reader with no prior context can understand what the system does from the purpose paragraph alone; every box on the container diagram has a one-line reason to exist; every NFR has a number and a way to check it; every ADR explains what was rejected, not only what was chosen; the risks section says something true and slightly uncomfortable, not "no known risks."

Stretch goals

- Present your design document to a colleague who's never seen it, and time how long it takes them to answer "what happens if this one container goes down?" A well-designed document makes that answer fast.
- Find a decision in your system that was never written down, interview whoever made it (or reconstruct the reasoning if they've left), and write the ADR retroactively.
- Redraw your container diagram as it would look in two years if current growth continues, and mark which containers would need to change first.

You've got it when...

- ☐ You can say, for any decision, whether it's architectural using the "hard to reverse, crosses boundaries, expensive to get wrong late" test.
 - ☐ You draw C4 diagrams at the zoom level the audience needs, and never mix levels on one diagram.
 - ☐ You can turn "make it fast/reliable/secure/cheap" into a number and a measurement method, for any system you're asked about.
 - ☐ You write ADRs that include what was rejected and why, not only what was chosen.
 - ☐ You can trace a real request across a container diagram and know it's accurate because you did.
 - ☐ You evaluate a redesign proposal by asking what it costs while it's happening, not only what it achieves when finished.
 - ☐ You can name the difference between fixing what's broken and rebuilding what merely looks untidy.
 - ☐ You resist over-engineering for a scale that doesn't exist yet, and resume-driven adoption of tools that don't solve a named problem.
-

Recap

- **Architecture is any decision that's hard to reverse and affects more than one part of a system** — not a title, a habit available to anyone whose choices reach beyond their own code.
 - **The C4 model** gives four fixed zoom levels — context, container, component, code — so a diagram matches what its reader actually needs to decide.
 - **Non-functional requirements** must become numbers ("p95 under 200ms," "99.5% success, measured monthly"), or they're wishes, not requirements — and every NFR needs a stated owner and measurement method.
 - **An ADR** captures a decision's context, the decision itself, the alternatives considered and rejected, and its consequences — so the reasoning outlives the meeting where it happened.
 - **The worked Riverstone platform** ties together Part V's data infrastructure and Part VI's AI systems into one described whole, with a traceable data flow and an honest risks section.
 - **Evolving a real system** means working within budget, headcount, and legacy constraints — migrating the lowest-risk piece first, running old and new side by side, and keeping a rollback path.
 - **The temptation to rewrite everything** after finally seeing the whole picture is common and usually wrong: fix what's actually broken, not what merely looks untidy on a new diagram.
-

Practice exercises

Warm-up

1. Using the three-property test in section 60.1, decide whether each is architectural: (a) renaming a Python variable; (b) choosing which cloud region to deploy in; (c) changing a chart's color; (d) choosing the message format two services use to talk to each other.
2. What's the difference between a level 1 (context) and level 2 (container) C4 diagram? Give an example of a question each one answers that the other doesn't.
3. Turn these into testable non-functional requirements: (a) "the dashboard should load quickly"; (b) "the system should be secure"; (c) "we shouldn't lose data."
4. List the six parts of an ADR from memory, in order.
5. Why does an ADR need an "alternatives considered" section? What goes wrong without one?

Core

6. Draw a context diagram (level 1) for a system you use or have built, showing at least three external people or systems.
7. Open one box from your context diagram into a container diagram. What are the deployable pieces, and how do they connect?
8. Write three non-functional requirements for that same system, each with a specific number and a stated way to measure it.

9. Trace one real request across your container diagram from end to end, the way section 60.5 traced a purchase-order email. Where, if anywhere, does the trace get vague or uncertain?
10. Write a full ADR for a real decision in a system you know, including at least two rejected alternatives.
11. Using the "nines" table in section 60.3, calculate the annual downtime allowed at 99%, 99.9%, and 99.99% availability. For a system you know, which level is actually appropriate, and why?
12. Riverstone's platform has four AI-adjacent containers that don't depend on each other (section 60.5). What's the architectural benefit of that independence, and what's given up by not sharing infrastructure between them?
13. A colleague proposes replacing Riverstone's orchestrator because a newer tool "looks better in the documentation." Using section 60.6's evolution pattern, what would you ask before agreeing?

Stretch

14. Write the component-level (level 3) diagram for the PO-intake pipeline, using Chapter 58's stages. What would a level 4 (code) diagram of the "validate" stage need to show, and why does this chapter recommend not drawing it by hand?
15. Design a migration plan for consolidating two duplicated pieces of infrastructure (like Meera's two model-serving setups in the story), following section 60.6's four-step pattern.
16. Riverstone's design document (section 60.5) lists three open risks. Propose how you'd investigate and close one of them, and what evidence would let you remove it from the risks list.

Think about it

17. A stakeholder asks you to skip writing the ADR because "we all agree, let's just build it." What do you say, and what would you be willing to compromise on (format, length) to get the decision recorded anyway?
18. Is a big-bang rewrite ever the right call? Under what specific conditions would section 60.6's "migrate the lowest-risk piece first" advice not apply?

Key terms

architecture · non-functional requirement (NFR) · functional requirement · C4 model · context diagram · container diagram · component diagram · code diagram · zoom level · architecture decision record (ADR) · system of record · availability ("nines") · freshness · trade-off · resume-driven design · big-bang rewrite · shadow mode (Chapter 56) · rollback path

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 61, Distributed Systems & Trade-offs:** what happens when the container diagram's boxes are on different machines — consistency, availability, and the failures Part V's stack has actually had.
 - **Chapter 62, Data Architecture Patterns:** naming and comparing the shapes (Lambda, Kappa, medallion, data mesh) that a warehouse-and-pipeline design like section 60.5's actually follows.
 - **Chapter 63, Automation Architecture & Governance:** the eight-container platform, inventoried, owned, and governed at company scale — building directly on the container diagram this chapter drew.
 - **Chapter 64, Security, Privacy, Governance & Responsible AI:** closing the "access control model not yet defined" risk this chapter's design document left open.
 - **Chapter 67, The Architect as Leader:** the conversation between Meera and Vikram in this chapter's story — influencing a decision without unilateral authority — done properly.
 - **Interview preparation:** the System Design Question Bank (Chapter 77) uses C4-style diagrams and NFR framing directly; "walk me through how you'd design X" is answered with this chapter's method.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. (a) Not architectural — trivial to reverse, affects nothing else. (b) Architectural — expensive to reverse (data residency, latency to every other system), affects everything deployed there. (c) Not architectural — costs nothing to change, affects one chart. (d) Architectural — every service on both ends has to agree on it, and changing it later means coordinating every consumer.
2. Context answers "what is this system for, and who does it talk to?" — the question a new employee or a board member asks. Container answers "what are its major moving pieces, and how do they connect?" — the question an engineer joining the team asks. A context diagram can't tell you which pipeline runs the ingestion; a container diagram crammed with every external person and system becomes unreadable.
3. (a) "The dashboard's initial view renders in under 2 seconds for 95% of loads, measured via the BI tool's own performance log." (b) "No credential can access data outside its assigned scope; access reviewed quarterly" (specific to whatever "secure" means for this system — always ask secure against what). (c) "Every write is retried with an idempotency key on failure, and reconciled against the source daily; zero unexplained row-count discrepancies, checked nightly."
4. Title and number; status and date; context; decision; alternatives considered; consequences.
5. Without it, the same debate happens again months later when someone new joins and asks "why don't we just use X?" — and nobody remembers that X was already considered and rejected, so the organization re-spends the time re-litigating a closed question, sometimes reversing it based on incomplete information the first decision-makers already had and accounted for.

6–10. Personal exercises with no single correct answer; check against the chapter's criteria — a context diagram with all arrows crossing the one central box; a container diagram whose pieces are genuinely independently deployable; NFRs with numbers and named measurement methods; a trace that follows one real request without gaps; an ADR with real, specific rejected alternatives.

11. 99%: about 3.65 days/year. 99.9%: about 8.75 hours/year. 99.99%: about 53 minutes/year. Appropriateness depends entirely on the system: a personal analytics dashboard rarely justifies 99.99% (the cost of achieving it — redundant infrastructure, on-call rotations, more complexity — usually exceeds the cost of the rare outage); a payment system or a safety alert often does.

12. Benefit: a failure in one (say, the defect model's serving endpoint going down) can't cascade into the others, and each can be redeployed, scaled, or even retired on its own schedule without coordinating with the other three teams. Cost: duplicated infrastructure (two separate serving setups, as the story found), which is wasted spend and operational overhead once there are enough of them to matter — exactly the trade-off Meera revisited by consolidating serving, while deliberately keeping the containers' logical independence.

13. Ask: what specific, named problem does the current orchestrator actually have that the new one solves? What would migration cost — in time, in running two systems in parallel, in retraining the team? Is there a real incident or limitation driving this, or only that the new tool "looks better"? If there's no named problem, this is resume-driven design (section 60.6's mistake table), and the answer is no, not yet — revisit if a real limitation appears.

14. A level 3 diagram of the PO-intake pipeline would show the seven Chapter 58 stages (read, extract, parse, validate, decide, write, report) as internal components, with arrows showing which stage calls which and where the exception queue branches off. A level 4 diagram of "validate" would need to show the actual validation functions and their logic — the chapter recommends against hand-drawing this because it changes with nearly every code commit, so a manually maintained version goes stale within weeks and becomes actively misleading, worse than no diagram at all.

15. Following section 60.6: (1) run both serving setups side by side for a defined period, comparing outputs on the same inputs; (2) migrate the lower-traffic, lower-risk model first (as Meera did with the defect model, not the busier support assistant); (3) keep the old serving path live and able to take traffic until the new one has run through at least one full realistic cycle; (4) write the ADR recording the consolidation decision, what was rejected (keeping them separate, or migrating the higher-traffic one first), and why.

16. For example, the "exception queue has no defined maximum wait time" risk: investigate by load-testing the confirmation step against a simulated festive-season email volume (using Chapter 58's real per-style accuracy and timing data), and set an explicit SLA once you know the real relationship between volume and queue length. The risk can be removed from the list once that SLA is defined, tested under realistic load, and either met or has a documented mitigation (more reviewers, an escalation path) for when it isn't.

17. Explain, briefly, what the ADR protects against — not disagreement now, but the same conversation happening again in eight months with nobody remembering why. Offer to write a genuinely short version (a title, one-sentence decision, and a two-line "why," which takes five minutes) rather than

insisting on the full six-part template every time — a short ADR that exists beats a thorough one that doesn't get written because it seemed like too much overhead.

18. A big-bang rewrite can be the right call when the existing system is actively unsafe to keep running (a security vulnerability with no incremental patch path), when it's built on infrastructure that's being discontinued with a hard deadline, or when the team and budget genuinely support running two full systems in parallel for the migration period. The "migrate lowest-risk piece first" advice doesn't apply when there's no way to run part of the old system and part of the new one at once — some technology changes are genuinely all-or-nothing — but this is rarer than most rewrite proposals assume, and the burden of proof should be on showing that a piecewise migration is truly impossible, not merely inconvenient.